MADHEVAN RAMALINGAM

WLAN Firmware & Driver Engineer — SoC Bring-up · Low-Power Architecture · Pre-Silicon Emulation
Chennai, India · madhevan1998@gmail.com · linkedin.com/in/madhevan-ramalingam · madironman.github.io

SUMMARY

Firmware engineer with 6+ years in embedded systems, most of them on 802.11 WLAN silicon: low-power SoC architecture, pre-silicon bring-up, host–firmware transports over PCIe, SDIO and on-die AXI, and now a Windows Wi-Fi 6/6E driver.

SKILLS

Wireless: IEEE 802.11a/b/g/n/ac/ax (Wi-Fi 6/6E), 802.11 power save (DTIM, PS-Poll, U-APSD, TWT), wake-on-WLAN and offloads, WPA2/WPA3 (CCMP, TKIP)
Firmware & OS: RTOS (FreeRTOS, ThreadX), CMSIS, bare-metal bring-up, Windows kernel-mode drivers (KMDF, WiFiCx, NetAdapterCx), embedded Linux (device tree, I2C)
SoC & interfaces: ARM Cortex-R4/M4/M52, AXI/AHB/APB, PCIe (D3, ASPM L1 substates, FLR), SDIO, DMA, UART, GPIO, PMU and power domains
Debug & validation: JTAG/ARM ADIv5, OpenOCD, GDB, WinDbg (KDNET), Driver Verifier, Lauterbach TRACE32, Veloce emulation, FPGA, RTL and waveform analysis
Languages & tools: C (primary); Python, TCL and C# for tooling; ARM assembly; Git, Jenkins CI, MSBuild/WDK

EXPERIENCE

embedUR Systems — Chennai, India **May 2020 – Present**
Technical Lead (2025–present) · Senior Software Engineer (2022–2025) · Software Engineer (2020–2022) · Intern (2020)

Client: Infineon — Windows Wi-Fi driver Jul 2026 – Present

- Main contributor to a Windows 11 WiFiCx driver for a Wi-Fi 6/6E PCIe chipset on ARM64, porting behaviour function by function from the Linux brcmfmac driver.
- Implemented Windows PCIe power management: firmware D3 handshake, hot resume without a firmware reload (cold-resume fallback), S0 idle, Modern Standby, and wake-on-WLAN with ARP/NS/GTK-rekey offload.
- Fixed a WPA2 4-way-handshake failure by tracing it to stale ARM64 cache lines on zero-copy TX buffers and cleaning the cache before DMA, which let zero-copy TX be switched back on.
- Added recovery from resets, surprise removal and firmware halts on two board designs (probe-time reset, FLR fallback, coredump and restart after a halt), and made teardown and the ring layer safe: timers, DPCs and work items flushed before free, and a sync-marker check before parsing ring entries.
- Built the wl.exe / dhd.exe diagnostics tools and GUI in C# on driver passthrough IOCTLs, and wrote the driver's board test suite and both project wikis.

Client: Synaptics — WLAN firmware and SoC bring-up Jun 2022 – Jul 2026

- Designed the sleep architecture of a combined MCU + Wi-Fi SoC: a four-state machine (MCU × WLAN, each awake/asleep) in which WLAN firmware moves from host buffers to local TCM buffers while the MCU sleeps, so either core sleeps without dropping traffic, with race-free entry and exit (level-held wake line, decision-time sleep veto, stuck-wake watchdog). Design signed off; implementation under way when the engagement closed.
- Designed the MCU–WLAN interrupt architecture of the on-die transport (PCIe msgbuf rings re-hosted on AXI): a FreeRTOS-safe NVIC driver on CMSIS and an instrumented test driver measuring latency, misses and spurious hits, which caught a clock-domain-crossing race on deassertion, fixed by deferring clear-pending.
- Let ~90% of the host-driver and transport development happen before first silicon by conceiving and designing a stand-in MCU platform: an existing combo chip's Bluetooth Cortex-M4 booted a FreeRTOS host image through a UART-patch and JTAG load chain and ran the transport against the WLAN Cortex-R4, with cross-core doorbells built from timer and chip-level interrupt registers.
- Cut WLAN test turnaround on the Veloce emulator from 8 h to ~30 min by running it headless with SSH access and building the full test-automation framework, taking emulator utilisation toward ~90% as it moved from bring-up-only use to nightly regressions and pre-merge checks; the team then released ROM for a tapeout in one week.
- Brought up JTAG/OpenOCD debug on the Veloce emulator, root-causing three RTL and place-and-route faults in the debug path and redesigning JTAG fetch for emulation speed, and added an OpenOCD backend so the verification teams' TCL scripts ran unmodified.
- Drove emulator bring-up of a new SoC from reset to ping, and contributed the architectural analysis behind a 2× transport throughput improvement and a shorter MCU boot.
- Ended zero-throughput stalls under sustained bidirectional SDIO traffic by diagnosing an RX pool-starvation loop from the pool counters and reserving a floor of packet pools for RX DMA refill.

- Hardened WPA2 without breaking mixed legacy networks by blocking pairwise TKIP while keeping group TKIP, enforced across the supplicant, host driver and firmware MAC key-install path.
- Implemented dynamic RAM-bank turn-off for a chip-level sleep-current reduction on an SDIO WLAN chipset, confining packet pools to the last bank and gating it only when every pool was idle; ported it to three earlier chipsets, designed follow-ons that also gate free memory during active operation, and shrank the firmware image 50–60% with a multistage download format.
- Kept the M2M DMA engine usable despite a silicon FSM defect: profiled it against memcpy to steer the hardware team to the root cause, and devised the runtime workaround that routes only the affected transfers through memcpy.
- Unblocked the SoC's deep-sleep mode despite a silicon RC-oscillator enable-latch defect, bringing the MCU's RC clock up via the WLAN core's internal JTAG master, then via an LDO overvolt applied only around sleep, and made the MCU an always-on wake source.
- Wrote bring-up debug tooling: a crash-triage engine that rebuilds system state from raw RAM dumps, a heap profiler, a struct-layout analyser, and a strap-pin kit that made remote CI reflash possible.
- Audited release binaries with a self-built disassembly scan for illegal undefined-instruction patterns, surfacing 200+ latent crash points.
- Resolved a day-0 field escalation by programming internal interrupt generators and an on-chip JTAG master to reach otherwise inaccessible debug registers.
- Mentored two engineers: an intern in C and how C programs map onto ARM memory, and a new power-save engineer in the hardware, the power-save mechanisms and the new SoC's power-save design.

Client: Hirschmann (Belden) — embedded Linux, industrial Ethernet switches May 2021 – Jun 2022

- Fixed a boot-time regression from a VxWorks-to-Linux port on two industrial-switch platforms, cutting Linux boot from 165–172 s to 40–45 s by tracing a 10 s per-slot retry loop in the hardware scan and moving module initialisation into two tasks.
- Brought up hot-pluggable SFP module support on three switch platforms, restoring module data and SFP-to-SFP traffic, by adding the missing device-tree entries and resolving I2C address collisions with a dummy-register scheme.

Client: CommScope (ARRIS) — cable-modem firmware and lab provisioning May 2020 – May 2021

- Debugged DOCSIS signal-strength, DHCP, Green Ethernet and public Wi-Fi hotspot (GRE tunnel) connection defects on SURFboard cable gateways across the SNMP and dmcli back ends.
- Cut client test-machine cost ~90% during a hardware shortage by replacing test laptops with Raspberry Pi clients and automating their allocation, so test scripts claimed any free Pi for iperf runs; one of five volunteers who brought up the lab's CMTS server.

EDUCATION

B.E. Electrical & Electronics Engineering — SRM Easwari Engineering College (Anna University), Chennai 2016 – 2020

- Department topper, 2016–2020 batch.
- Thesis internship, Ericsson Research India (Dec 2019 – Mar 2020): a TV-white-space surveillance link transmitting off a Raspberry Pi GPIO pin and receiving on an RTL-SDR; published by Springer (ICIDCA, 2021).

INVENTION DISCLOSURE

- Co-inventor on a filed invention disclosure in 802.11 power save: station-side, controlled retrieval of AP-buffered packets (hybrid U-APSD / PM-2 with adaptive control) for hosts with long wake latency.